
***IoT BASED SMART TROLLEY USING RFID TAGS, FIREBASE
DATABASE AND BLE BEACON NAVIGATION***

TESTING AND VALIDATION REVIEW(TVR)

Smart IoT-Based Shopping Trolley

Here is a PDR/TVR-ready version based on your project using ESP32 + RFID + Firebase + BLE Beacon Navigation.

1. Functional Testing Report

The prototype was tested module-by-module and then as an integrated system.

Module	Test Performed	Expected Result	Status
ESP32	Upload and execute program	ESP32 operates correctly	Passed
RFID Reader	Scan registered RFID tags	Correct tag ID detected	Passed
Product Database	Match RFID ID with Firebase data	Product name and price retrieved	Passed
Price Calculation	Add multiple products	Total price calculated correctly	Passed
Display	Show product and price	Information displayed clearly	Passed
Buzzer	Product detection feedback	Buzzer activates correctly	Passed
IR Sensor	Detect item movement	Item presence detected	Passed
Firebase	Send/retrieve product data	Data synchronized with cloud	Passed
BLE Beacon	Detect beacon signal	Beacon detected by trolley ESP32	Passed
BLE RSSI	Measure signal strength	RSSI value received for proximity estimation	Passed
Integrated System	Test complete trolley operation	Modules work together	Under Testing

2. Performance Evaluation

The performance of the prototype was evaluated based on RFID detection, Firebase communication, BLE navigation and system response.

Parameter	Evaluation
RFID Detection	Successfully detects registered RFID tags within the reader's operating range.
Product Identification	RFID UID is matched with the corresponding product information.
Price Calculation	Product prices are added to generate the running shopping total.
Firestore Response	Product information can be stored and retrieved through Wi-Fi.
BLE Detection	Trolley ESP32 detects the designated BLE navigation beacon.
RSSI Measurement	RSSI values are obtained to estimate the trolley's proximity to the beacon.
Display Response	Product and price information is displayed after successful identification.
System Response	The ESP32 responds to sensor inputs and performs the programmed operations.

Overall evaluation:

The prototype successfully demonstrates the major functions required for the smart trolley. Further calibration is required for accurate BLE distance estimation and reliable operation in different indoor environments.

3. Test Cases

Test Case	Input	Expected Output	Result
TC01	Power ON ESP32	System starts successfully	Pass
TC02	Present valid RFID tag	Product is identified	Pass
TC03	Present unknown RFID tag	Unknown/unregistered tag message	Pass
TC04	Scan multiple products	All products are added	Pass

Test Case	Input	Expected Output	Result
TC05	Add product	Correct price shown	Pass
TC06	Remove product	Product/total updated	Under Testing
TC07	Connect to Wi-Fi	ESP32 connects to network	Pass
TC08	Request product data	Firestore returns correct data	Pass
TC09	Disconnect Wi-Fi	System detects connection failure	Under Testing
TC10	BLE beacon nearby	Beacon detected	Pass
TC11	Move trolley away from beacon	RSSI decreases	Pass
TC12	Display product information	Correct information displayed	Pass
TC13	Multiple BLE beacons	Required beacon can be identified	Under Testing
TC14	Continuous operation	System operates without unexpected restart	Under Testing

4. Error Analysis

During prototype testing, the following possible errors were identified:

RFID Errors

- RFID tag may not be detected if it is outside the reader's effective range.
- Multiple tags close to the reader may cause reading conflicts.
- Metal objects can affect RFID performance.

Firestore/Wi-Fi Errors

- Internet/Wi-Fi failure can temporarily prevent database communication.
- Incorrect Firestore credentials or database paths can cause data retrieval errors.
- Network delay can increase response time.

BLE Errors

- RSSI varies because of walls, people, trolley materials and other obstacles.
- RSSI-based distance estimation is approximate rather than exact.
- Multiple BLE beacons can produce overlapping signals.

Hardware Errors

- Loose jumper wires can cause intermittent operation.
- Insufficient power can cause ESP32 resets or unstable peripherals.
- Sensor alignment can affect detection reliability.

5. Debugging Report

Problem	Possible Cause	Debugging Method	Solution
RFID not detecting	Wiring/range issue	Checked SPI connections and tag position	Corrected wiring and reader position
Display blank	Incorrect connections/address	Checked power and communication	Corrected wiring/address
Firebase not updating	Wi-Fi/network issue	Checked serial monitor and connection	Re-established Wi-Fi and verified database configuration
Incorrect product information	RFID UID mismatch	Compared UID with database	Updated RFID-product mapping
BLE beacon not detected	Beacon configuration/scanning issue	Checked BLE scan results	Verified beacon name and scanning configuration
Unstable BLE distance	RSSI fluctuations	Observed RSSI at different positions	Added filtering/averaging
ESP32 restarting	Power instability	Monitored power supply	Improved power connection
Buzzer not working	Incorrect GPIO/wiring	Tested GPIO separately	Corrected connection/code

6. Power Consumption Analysis

The smart trolley consists of an ESP32, RFID reader, display, sensors, buzzer and BLE functionality.

Major power-consuming components

Component	Power Requirement
ESP32 DevKit	Main controller and Wi-Fi/BLE operation
RFID Reader	Active during RFID scanning
LCD/OLED	Continuous display operation
IR Sensor	Continuous/periodic sensing
BLE Beacon ESP32	Continuous BLE advertising
Buzzer	Consumes power only when activated
LEDs	Low power, depending on resistor/current

Power Analysis

The prototype is designed to operate from a **USB power bank**. The ESP32 can receive power through its USB power input, making the system portable and suitable for trolley operation.

Power-saving methods:

- Use sleep/low-power modes when appropriate.
- Reduce unnecessary display updates.
- Scan BLE at controlled intervals instead of continuously at maximum frequency.
- Activate the buzzer only when required.
- Use efficient voltage regulation and proper power distribution.

For the final report, actual current and battery-runtime values should be measured using a USB power meter or multimeter rather than estimated.

7. Reliability Testing

Reliability testing is performed to verify that the trolley continues to operate correctly during repeated use.

Tests performed/planned

1. **Repeated RFID Test** – Scan the same RFID tag multiple times and verify consistent identification.
2. **Multiple Product Test** – Add several products sequentially and verify the correct total.
3. **Continuous Operation Test** – Run the ESP32 for an extended period and check for unexpected resets.
4. **Wi-Fi Recovery Test** – Disconnect and reconnect Wi-Fi and verify that the system recovers.
5. **BLE Test** – Move the trolley between different locations and verify beacon detection.
6. **Power Test** – Operate the trolley using a power bank and observe stability.
7. **Integrated Test** – Operate RFID, Firebase, display and BLE functions together.

Reliability objective:

The system should consistently identify registered products, maintain correct shopping information and provide stable BLE navigation without unexpected

8. Future Improvements

Hardware Improvements

- Design and fabricate a **custom PCB** instead of breadboard wiring.
- Use a stronger and more compact enclosure for electronics.
- Improve RFID reader positioning for reliable product detection.
- Add a better rechargeable power system with battery-level monitoring.
- Add additional BLE beacons for improved indoor navigation accuracy.

Software Improvements

- Develop a dedicated mobile/web application for customers.
- Add real-time shopping-cart synchronization with Firebase.
- Implement automatic removal detection.
- Add user authentication and secure Firebase rules.
- Add offline operation when Wi-Fi is temporarily unavailable.

- Improve BLE RSSI filtering and navigation algorithms.

Payment Improvements

- Generate a **QR code after shopping is completed**.
- Display the QR code on a suitable display or provide it through a mobile application.
- Integrate a secure digital payment gateway in the future.

Navigation Improvements

- Use multiple BLE beacons for zone/location identification.
- Implement trilateration or fingerprinting instead of relying only on a single RSSI value.
- Add voice or visual turn-by-turn navigation.
- Provide directions such as "**Move straight**", "**Turn left**", "**Turn right**".

Final Goal

The final version aims to provide a **fully integrated smart shopping trolley** where:

RFID Tag → Product Identification → Firebase → Price Calculation → Display → QR Payment

and

BLE Beacons → ESP32 → Indoor Position/Proximity → Navigation Guidance